

Name: Sarulatha D

ACE ID: ACE12402

Date: 28.01.2025

Azure Scenario's Task

Scenario 1: Your team needs to deploy a virtual machine in azure CLI to test a new software application. The team has requested both windows and linux VM. How could you set up these virtual machines and what considerations are needed for pricing and OS licensing.

Steps to create virtual machine:

1. Go to [Home - Microsoft Azure](#), and on the left sidebar, select Virtual Machines.
2. Click on + Add to create a new VM.
3. Choose a subscription.
4. Select a Resource Group.
5. **Configure VM settings:**
 - Give the virtual machine name.
 - Select the region.
 - Select the availability Options.
 - Image: Select the OS (Windows, Linux, etc.).
 - Size: Choose the size (CPU, RAM).
 - Authentication type: Choose SSH key (Linux) or password (Windows).
 - Inbound port rules: Select which ports we want to open (e.g., SSH (Secured Shell) for Linux, RDP (Remote Desktop Protocol) for Windows).
6. **Storage Configuration**

Azure VMs use managed disks for storage. Choose between Standard SSD, Premium SSD, or Standard HDD depending on performance needs.
7. **Networking**

Choose an existing Virtual Network or create a new one.
8. **Review and Create**

Review all the settings and hit Create to deploy the VM.

Pricing Considerations:

Prices may vary based on

- VM size (CPU, RAM) and type (general-purpose, compute-optimized or memory-optimized).
- Operating system: windows (charged for both the compute resource and the Windows license) or linux (mostly free)

- Licensing models: pay as you go, reserved instances, azure hybrid benefit.
- Storage cost: type (Standard SSD, Premium SSD, or Standard HDD) and disk size (128GB, 512GB)

Scenario 2: The IT security team has requested the sensitive data stored in azure storage account we encrypted to meet compliance. How could you ensure the data stored in azure is encrypted, and what encryption types are available?

Storage Service Encryption (SSE) uses 256-bit Advanced Encryption standard which handles encryption, decryption, and key management. Here's how we can set it up and the types of encryptions available:

Setting Up Encryption in Azure Storage

1. **Sign in to Azure:** Go to the Azure portal and sign in with the credentials.
2. **Navigate to the Storage Account:** In the Azure portal, search for "Storage accounts" and select the storage account we want to encrypt.
3. **Configure Encryption:**
 - a. Under the "Settings" section, select "Encryption".
 - b. By default, Azure Storage encrypts data at rest using Microsoft-managed keys. We can continue using these keys or opt for customer-managed keys.
 - c. To use customer-managed keys, select "Customer-managed keys" and specify the key vault and key to use. The key vault must have purge protection enabled.

Types of Encryptions Available

1. **Service-Side Encryption (SSE):**
 - a. **Microsoft-Managed Keys:** Azure handles the encryption and key management for us. This is the default option and is suitable for most scenarios.
 - b. **Customer-Managed Keys:** we manage our own encryption keys using Azure Key Vault. This provides greater control over key management and rotation.
 - c. **Infrastructure Encryption:** Adds a second layer of encryption using a different set of keys managed by Azure. This is useful for highly sensitive data.
2. **Client-Side Encryption:**

Data is encrypted by the client application before it is sent to Azure Storage. This ensures that data is encrypted end-to-end and that only the client has access to the encryption keys

Scenario 3: You are responsible for setting up devops pipeline in azure devops for your application. The pipeline must deploy code to azure app service and notify the team if the

deployment fails. How could you configure this pipeline to meet the requirements. How team notify the fails.

To set up a DevOps pipeline in Azure DevOps for deploying the application to Azure App Service and notifying the team in case of deployment failure, here are the steps to follow:

1. Create a Pipeline in Azure DevOps

- Go to **Azure DevOps**.
- Navigate to **Pipelines** and create a new pipeline.
- Choose the repository and select the appropriate pipeline configuration (e.g., YAML or classic editor).

2. Set up Deployment to Azure App Service

3. Set up Notifications for Failure

To notify the team if the deployment fails, we can configure a **failure notification** in Azure DevOps. This can be done using Azure DevOps **Service Hooks** or **Email Notifications**.

1. Using Email Notifications

- (1) Navigate to **Project Settings** in Azure DevOps project.
- (2) Go to **Notifications**.
- (3) Add a new notification.
- (4) Configure it to trigger on **Pipeline Failure**:
 - a. **Event**: Build/Release Failed (or Pipeline Failed)
 - b. **Condition**: set it to trigger when the pipeline fails.
 - c. **Subscriber**: select specific team members, groups, or a distribution list.
- (5) Customize the notification message if needed and save it.

This way, whenever the deployment fails, an email will be sent to the team.

2. Using Service Hooks (Slack, Teams, etc.)

If we prefer real-time notifications in tools like Slack or Microsoft Teams, we can use **Service Hooks**.

- (I) Go to **Project Settings > Service Hooks**.
- (II) Click on **Create Subscription** and choose the service (Slack, Teams, etc.) we want to integrate.
- (III) Configure the trigger for the subscription to be a **Pipeline Failure**.
- (IV) Set up the service hook to send a message to the relevant channel or team.

This can be configured for instant notifications to team communication channels.

In this case, if the deployment fails, the team will receive an email notifying them of the failure.

This setup covers the deployment to Azure App Service and notifications if the deployment fails, ensuring the team is always aware of any issues during the deployment process.

Scenario 4: Your organization is moving its on premises SQL database to azure. The database must remain accessible during migration with minimal downtime. Which azure service would you use, and how could you perform the migration?

Azure database migration service -- This service is designed to help with seamless database migrations with minimal downtime, including moving databases to Azure SQL Database or Azure SQL Managed Instance.

Here's how to perform the migration:

1. Preparation:

- a. Ensure on-premises SQL Server environment is supported and meets any prerequisites for migration
- b. Prepare the target Azure environment, which could be an Azure SQL Database, an Azure SQL Managed Instance, or an SQL Server on an Azure VM.
- c. Set up the necessary networking configuration to allow communication between the on-premises server and Azure.

2. Use Azure Database Migration Service (DMS):

- a. **Create a Migration Project:** In the Azure portal, create a migration project using Azure DMS.
- b. **Select the Source and Target:** Choose on-premises SQL Server as the source and the desired Azure SQL service as the target.
- c. **Assess Compatibility:** DMS will allow us to assess the compatibility of the source database with the target environment, helping identify potential issues.

3. Configure the Migration:

- a. **Choose the Migration Method:** choose between a one-time migration or continuous sync to allow for minimal downtime. The continuous sync option uses DMS to replicate ongoing changes from the source to the target database during the migration process.
- b. **Start Initial Migration:** The initial data migration can be done with minimal downtime, as we migrate the bulk of the data first.

4. Cutover to Azure:

- a. Once the initial migration is complete and the target database is up to date, a cutover can be scheduled. During the cutover, any final changes made to the on-premises database will be applied to the Azure database.

- b. After the cutover, the database will be fully hosted in Azure.
- 5. **Post-Migration Testing:**
 - a. Validate that the application is functioning as expected with the Azure-hosted database.
 - b. Monitor the performance and resource usage of the new Azure database and adjust if needed.

By using Azure DMS with continuous replication, the migration process minimizes downtime as the database remains accessible during the entire migration, with final cutover being the only period of downtime.